

学习的内容及流程

爬虫基础

字符串相关概念
HTTP/HTTPS
HTTP请求过程
常用请求方法
请求头常见参数
响应状态码
Chrome抓包工具

requests

GET请求
POST请求
使用代理
session

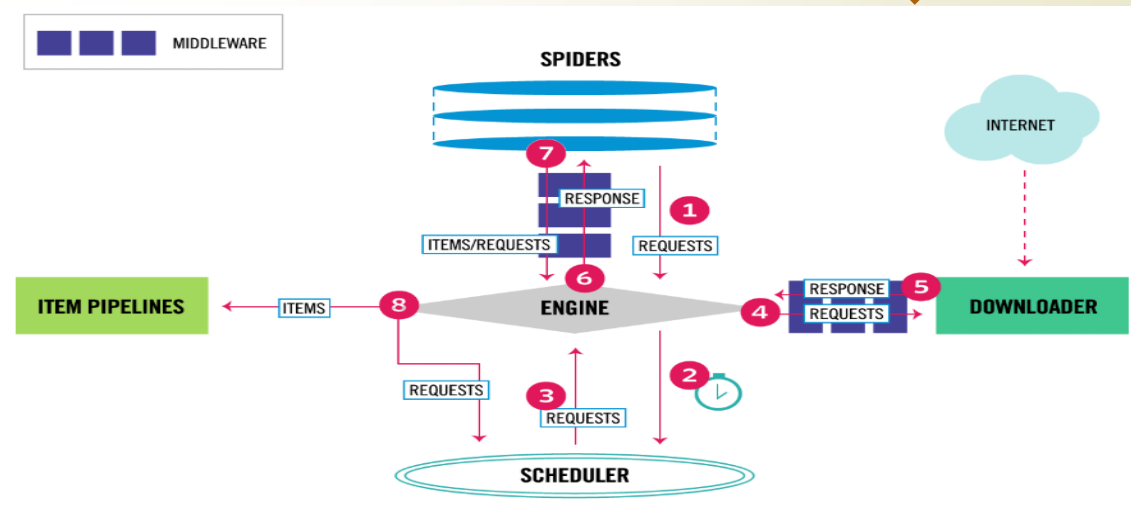
解析库

lxml库
BeautifulSoup
正则表达式

数据存储

JSON文件
CSV文件

Scrapy 框架



二、requests 请求库

号称 “HTTP for humans”



安装和文档地址

进入到需要安装的环境中，运行

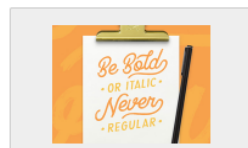
`pip install requests`

github地址: <https://github.com/psf/requests>

中文文档: http://docs.python-requests.org/zh_CN/latest/index.html



Requests is an elegant and simple HTTP library for Python, built for human beings.



Requests: 让 HTTP 服务人类

发行版本 v2.18.1. (安装说明)

license Apache 2.0 wheel yes python 2.7 | 3.5 | 3.6 | 3.7 | 3.8 | 3.9 codecov unknown Say Thanks! 🙏

Requests 唯一的一个非转基因的 Python HTTP 库，人类可以安全享用。

警告：非专业使用其他 HTTP 库会导致危险的副作用，包括：安全缺陷症、冗余代码症、重新发明轮子症、啃文档症、抑郁、头疼、甚至死亡。

看吧，这就是 Requests 的威力：

```
>>> r = requests.get('https://api.github.com/user', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.text
u'{"type": "User"...}'
>>> r.json()
{'private_gists': 419, 'total_private_repos': 77, ...}
```



发送GET请求



requests.get()发送get请求

```
import requests
response = requests.get('http://www.baidu.com/')
if response.status_code == 200:
    print('获得正常响应!')
```

在不设置请求头参数的情况下

```
for key, value in response.request.headers.items():
    print(key, ': ', value)
```



```
User-Agent : python-requests/2.20.0
Accept-Encoding : gzip, deflate
Accept : */*
Connection : keep-alive
```

正常的请求头

```
user-key=b10bdd6a-ef32-49ed-8685-c56c455783ea; pinId=vnu9UPVngs_zm8JxgwZ1BrV9-x-f3wJ7; p
p50q4oGa18F11VbxXvdPyGyg0H244%3D; _pst=jd_7095abebe6144; cn=101; __jdv=76161171|direct|-
kID=10ncvvhT4-cE1YdgtSL4pahOmxFb-wwQNPiHDP3pC2hptuxcJi8hxoI1H4a1VUreJo0P1kYHLAJxf-DktLaG
shshshfp=33641408df393ad1b8d7e482748fe2b0; shshshsID=a7d729b64e018adf84f9c515e409cb5e_1_
7.1616465504.1616486091.24; __jdb=122270672.2.161468791666818187451|24.1616486091; __jdc
4MIWFCUKIUQFVBWNGJNM4SULRG754LHK7EVHFI2DSX2RCBYDKSNWBYQE
Host: shouji.jd.com
Pragma: no-cache
Referer: https://www.jd.com/
sec-ch-ua: "Google Chrome";v="89", "Chromium";v="89", ";Not A Brand";v="99"
sec-ch-ua-mobile: ?0
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-site
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
```



我是爬虫，
封我呀笨蛋！



添加headers参数

```
import requests
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/'
                '537.36(KHTML, like Gecko) Chrome/88.0.4324.190 Safari/537.36'
}
response = requests.get('https://shouji.jd.com/', headers=headers)
for key, value in response.request.headers.items():
    print(key, ': ', value)
```

Connected to pydev debugger (build 202.6397.98)

User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36(KHTML, like Gecko) Chrome/88.0.4324.190 Safari/537.36

Accept-Encoding : gzip, deflate

Accept : */*

Connection : keep-alive

user-agent网址:

<http://www.useragentstring.com/pages/useragentstring.php?typ=Browser>



response的一些属性

1. response.text: 响应内容, 为网页的html代码, str类型

```
print(response.text)
```



```
<!DOCTYPE html>
<html>
<head>
  <meta charset="gbk" />
  <title>手机通讯频道</title>
  <meta name="keywords" content="" />
  <meta name="description" content="" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
  <meta name="renderer" content="webkit" />
  <!--[if lte IE 8]>
    <script src="//misc.360buyimg.com/mtd/pc/index/home/ie6tip.min.js"></script>
  <![endif]-->
  <link rel="dns-prefetch" href="//static.360buyimg.com">
  <link rel="dns-prefetch" href="//misc.360buyimg.com">
  <link rel="dns-prefetch" href="//img10.360buyimg.com">
  <link rel="dns-prefetch" href="//img11.360buyimg.com">
  <link rel="dns-prefetch" href="//img12.360buyimg.com">
  <link rel="dns-prefetch" href="//img13.360buyimg.com">
  <link rel="dns-prefetch" href="//img14.360buyimg.com">
  <link rel="dns-prefetch" href="//img20.360buyimg.com">
  <link rel="dns-prefetch" href="//img30.360buyimg.com">
  <link rel="dns-prefetch" href="//ai.jd.com">
  <link rel="icon" href="//www.jd.com/favicon.ico" mce_href="//www.jd.com/favicon.ico" type="image/x-icon">
</script>
!function (n) {
```

输出结果

```
view-source:https://shouji.jd.com
自动换行
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="gbk" />
5   <title>手机通讯频道</title>
6   <meta name="keywords" content="" />
7   <meta name="description" content="" />
8   <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1" />
9   <meta name="renderer" content="webkit" />
10  <!--[if lte IE 8]>
11    <script src="//misc.360buyimg.com/mtd/pc/index/home/ie6tip.min.js"></script>
12  <![endif]-->
13  <link rel="dns-prefetch" href="//static.360buyimg.com">
14  <link rel="dns-prefetch" href="//misc.360buyimg.com">
15  <link rel="dns-prefetch" href="//img10.360buyimg.com">
16  <link rel="dns-prefetch" href="//img11.360buyimg.com">
17  <link rel="dns-prefetch" href="//img12.360buyimg.com">
18  <link rel="dns-prefetch" href="//img13.360buyimg.com">
19  <link rel="dns-prefetch" href="//img14.360buyimg.com">
20  <link rel="dns-prefetch" href="//img20.360buyimg.com">
21  <link rel="dns-prefetch" href="//img30.360buyimg.com">
22  <link rel="dns-prefetch" href="//ai.jd.com">
23  <link rel="icon" href="//www.jd.com/favicon.ico" mce_href="//www.jd.com/favicon.ico" type="image/x-icon">
24  </script>
25  !function (n) {
```

右键→查看网页源代码

三、解析库

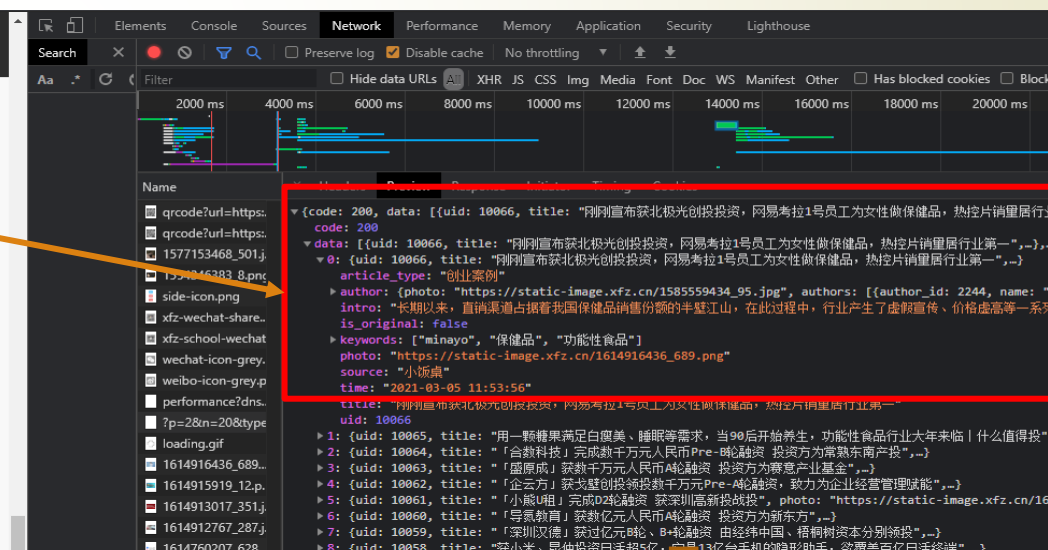


爬虫中数据的分类

根据爬虫爬取数据的类型来有规律的提取和解析数据，以小饭桌为例：



结构化数据



response.json()

直接转化为python类型 (Dict)

结构化数据 (JSON)



非结构化数据

```
<!DOCTYPE html>
<html>
  <head>
    <title>小饭桌</title>
    <meta name="keywords" content="小饭桌,不凡商业,小饭桌创业课堂,不凡人物,小饭桌创业课堂,小饭桌A轮融资,小饭桌小饭局,小饭桌创业,融资,融资服务,融资加速,
    桌友,创业礼包,直播课堂,在线课堂,小饭桌夜校,凡卓资本">
    <meta name="description" content="小饭桌是国内领先的创业创新服务平台。旗下包含小饭桌媒体和小饭桌创业课堂,链接中国新锐的创业创新人群,帮助创业者成就
    就不凡的事业。">
    <meta charset="utf-8">
    <meta content="webkit" name="renderer">
    <meta content="user-scalable=no,width=device-width,initial-scale=1.0" name="viewport">
    <link rel="icon" href="/static/build/images/favicon.ico" type="image/x-icon">
    <meta content="小饭桌" name="apple-mobile-web-app-title">
    <meta name="format-detection" content="telephone=no">
    <meta name="apple-touch-fullscreen" content="yes">
    <meta content="pc,mobile" name="applicable-device">
    <meta content="yes" name="apple-mobile-web-app-capable">
    <meta content="black" name="apple-mobile-web-app-status-bar-style">
    <meta content="True" name="HandheldFriendly">
    <meta content="320" name="MobileOptimized">
    <meta name="baidu-site-verification" content="oRCR63shm4">
    <meta name="360-site-verification" content="ce749d5c23b02abe3037f0a236f2fd66">
    <meta name="sogou_site_verification" content="1x9F1mhAoZ">
    <meta name="google-site-verification" content="MQPNG2F-SLWYWkUSuadD35Pf-Yhi2P1J0Az1pXv092k">
    <meta property="wb:webmaster" content="ae3864f5498fdd13">
    <meta property="og:type" content="webpage">
    <meta property="og:url" content="https://www.xfz.cn/" id="ogUrl">
    <meta property="og:title" content="小饭桌" id="ogTitle">
    <meta property="og:description" content="小饭桌是国内领先的创业创新服务平台。旗下包含小饭桌媒体和小饭桌创业课堂,链接中国新锐的创业创新人群,帮助
    创业者成就不凡的事业。" id="ogDes">
    <meta property="og:image" content="https://static-image.xfz.cn/xfz-share-logo-300.png" id="ogImg">
```



非结构化数据 (HTML)



正则表达式、lxml库、bs4库
解析并提取需要的内容



Xpath语法和lxml库

XPath (XML Path Language) 是一门在 HTML/XML 文档中查找信息的语言，可用在 HTML/XML 文档中对元素和属性进行遍历

官方文档: <https://www.w3school.com.cn/xpath/index.asp>



lxml是一款高性能的 Python HTML/XML 解析器，我们可以利用 XPath，来快速的定位特定元素以及获取节点信息



选取节点

XPath使用路径表达式来选取HTML文档中的节点或者节点集

```
<html>
  <head>
    <meta charset="UTF-8">
    <title>XPath演示</title>
  </head>
  <body>
    <div>
      <ul>
        <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
        <li class="item2"><a href="https://www.jd.com"> 京东商城</a></li>
        <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
        <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
        <li class="item5"><a href="https://www.google.com">谷歌</a></li>
      </ul>
    </div>
  </body>
</html>
```

/html: 选取根节点html 注意: /head会报错

```
html = etree.HTML(text)
htmlTags = html.xpath('/html')
print(etree.tostring(htmlTags[0], encoding='utf-8').decode('utf-8'))
```

head: 选取head节点 body: 选取body节点

```
headTags = html.xpath('head')
print(etree.tostring(headTags[0], encoding='utf-8').decode('utf-8'))
```

```
bodyTags = html.xpath('body')
print(etree.tostring(bodyTags[0], encoding='utf-8').decode('utf-8'))
```

注意: xpath('div')或xpath('li')报错

通过路径选取所有的li标签

```
liTags = html.xpath('body/div/ul/li')
liTags = html.xpath('//div/ul/li')
liTags = html.xpath('//ul/li')
liTags = html.xpath('//li')
for liTag in liTags:
    print(etree.tostring(liTag, encoding='utf-8').decode('utf-8'))
```

//: 从全局节点中选择, 不管位置在哪

/: 从body节点中选取名为div的儿子节点

```
divsTags = html.xpath('body/div')
print(etree.tostring(divsTags[0], encoding='utf-8').decode('utf-8'))
divsTags = html.xpath('//div')
print(etree.tostring(divsTags[0], encoding='utf-8').decode('utf-8'))
```



节点中的谓词

注意：经过xpath后返回的是一个列表！

谓词用来查找某个特定的节点或者包含某个指定值的节点，被嵌在方括号中

```
<html>
  <head>
    <meta charset="UTF-8">
    <title>XPath演示</title>
  </head>
  <body>
    <div>
      <ul>
        <li class="item1"><a href="https://geekori.com">geekori.com</a></li>
        <li class="item2"><a href="https://www.jd.com">京东商城</a></li>
        <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
        <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
        <li class="item5"><a href="https://www.google.com">谷歌</a></li>
      </ul>
    </div>
  </body>
</html>
```

选取class属性值为item2的li标签

```
liTag = html.xpath('//ul/li[@class="item2"]')
print(etree.tostring(liTag[0], encoding='utf-8').decode('utf-8'))
```

选取class属性值为item2的li标签中的href属性

```
href = html.xpath('//ul/li[@class="item2"]//@href')
print(href)
```

//ul/li[1]: 选取div下第一个li节点（下标1开始）

```
liTag = html.xpath('//ul/li[1]')
print(etree.tostring(liTag[0], encoding='utf-8').decode('utf-8'))
```

//ul/li[last()]: 选取div下最后一个li节点

```
liTag = html.xpath('//ul/li[last()]')
print(etree.tostring(liTag[0], encoding='utf-8').decode('utf-8'))
```

//ul/li[position()>3]: 选取div下下标大于3的li节点

```
liTags = html.xpath('//ul/li[position()>3]')
for liTag in liTags:
    print(etree.tostring(liTag, encoding='utf-8').decode('utf-8'))
```

选取class属性值为item2的li标签中的文本

```
href = html.xpath('//ul/li[@class="item2"]//text()')
print(href[0].strip())
```



lxml库及安装

lxml是一个HTML/XML的解析器，主要功能是解析和提取HTML/XML数据

官方文档：<http://lxml.de/index.html>

The screenshot shows the lxml.de website. The browser address bar displays 'lxml.de/index.html'. The page has a green header with the text 'Like the tool? Help making it better! Your donation helps!' and a small green snake icon. The left sidebar contains a table of contents with links to various sections. The main content area features the title 'lxml - XML and HTML with Python' and a description of the library. It also includes sections for 'Introduction' and 'Support the project'.

Like the tool? Help making it better! Your donation helps!

lxml

- o lxml
 - Introduction
 - Support the project
 - Documentation
 - Download
 - Mailing list
 - Bug tracker
 - License
 - Old Versions
 - Project income report
 - Legal Notice for Donations
- o Why lxml?
- o Installing lxml
- o Benchmarks and Speed
- o ElementTree compatibility of lxml.etree
- o lxml FAQ - Frequently Asked Questions

Developing with lxml

- o The lxml.etree Tutorial
- o API reference
- o APIs specific to lxml.etree
- o Parsing XML and HTML with lxml
- o Validation with lxml
- o XPath and XSLT with lxml
- o lxml.objectify
- o lxml.html
- o lxml.cssselect
- o BeautifulSoup Parser
- o html5lib Parser

Extending lxml

- o Document loading and URL resolving
- o Python extensions for XPath and XSLT
- o Using custom Element classes in lxml
- o Sax support
- o The public C-API of lxml.etree

Developing lxml

- o How to build lxml from source
- o How to read the source of lxml
- o Release Changelog
- o Credits

Sitemap

Like the tool? Help making it better! Your donation helps!

lxml - XML and HTML with Python

lxml is the most feature-rich and easy-to-use library for processing XML and HTML in the Python language.

» Introduction

The lxml XML toolkit is a Pythonic binding for the C libraries **libxml2** and **libxslt**. It is unique in that it combines the speed and XML feature completeness of these libraries with the simplicity of a native Python API, mostly compatible but superior to the well-known **ElementTree** API. The latest release works with all CPython versions from 2.7 to 3.9. See the **introduction** for more information about background and goals of the lxml project. Some common questions are answered in the **FAQ**.

» Support the project

lxml has been downloaded from the **Python Package Index** millions of times and is also available directly in many package distributions, e.g. for Linux or macOS.

Most people who use lxml do so because they like using it. You can show us that you like it by blogging about your experience with it and linking to the project website.

If you are using lxml for your work and feel like giving a bit of your own benefit back to support the project, consider sending us money through GitHub Sponsors, Tidelift or PayPal that we can use to buy us free time for the maintenance of this great library, to fix bugs in the software, review and integrate code contributions, to improve its features and documentation, or to just take a deep breath and have a cup of tea every once in a while. Please read the Legal Notice below, at the bottom of this page. Thank you for your support.

Support lxml through **GitHub Sponsors**

安装：`pip install lxml`



lxml的基本使用

```
from lxml import etree

text = '''
    <ul>
      <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
      <li class="item2"><a href="https://www.jd.com"> 京东商城</a>
      <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
      <li class="item4" value="1234"><a href="https://www.microsoft.com">微软
      <li class="item5"><a href="https://www.google.com">谷歌</a></li>
    </ul>
'''

html = etree.HTML(text)
print(etree.tostring(html, encoding='utf-8').decode('utf-8'))
```



对不规范HTML代码的自动补全

```
<html><body><ul>
  <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
  <li class="item2"><a href="https://www.jd.com"> 京东商城</a>
</li><li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
  <li class="item4" value="1234"><a href="https://www.microsoft.com">微软
  <li class="item5"><a href="https://www.google.com">谷歌</a></li>
</a></li></ul>
</body></html>
```

- ✓ 通过etree中的HTML方法对HTML字符串进行解析，返回的html是一个Element html类型数据
- ✓ 运用etree.tostring方法展示Element html类型数据
- ✓ etree.tostring方法返回的是bytes类型的数据，需要用utf-8来解码



总结lxml结合xpath使用的注意事项

1. 使用Element.xpath方法来执行xpath的选择，示例代码如下：

```
liTag = html.xpath('//ul/li[1]')
```

2. 获取某个标签的属性，示例代码如下：

```
href = html.xpath('//ul/li[@class="item2"]/@href')
```

3. 获取文本，是通过xpath中的text()函数，示例代码如下：

```
href = html.xpath('//ul/li[@class="item2"]//text()')
```

4. 在某个标签下，再执行xpath函数，获取这个标签下的子孙元素那么应该在斜杠之前加一个`.`，代表是在当前元素下获取，示例代码如下：

```
ulTag = html.xpath('//ul')[0]
aTag1 = ulTag.xpath('//a')[0]
print(etree.tostring(aTag1, encoding='utf-8').decode('utf-8'))
print('=' * 40)
aTag2 = ulTag.xpath('.//a')[0]
print(etree.tostring(aTag2, encoding='utf-8').decode('utf-8'))
```

```
<a href="https://www.taobao.com">咸鱼</a>
=====
<a href="https://geekori.com">geekori.com</a>
```

注意：经过xpath后返回的是一个列表！

```
<html>
<head>
  <meta charset="UTF-8">
  <li class="item2"><a href="https://www.taobao.com">咸鱼</a></li>
  <title>XPath演示</title>
</head>
<body>
  <div>
    <ul>
      <li class="item1"><a href="https://geekori.com">geekori.com</a></li>
      <li class="item2"><a href="https://www.jd.com">京东商城</a></li>
      <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
      <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
      <li class="item5"><a href="https://www.google.com">谷歌</a></li>
    </ul>
  </div>
</body>
</html>
```




BeautifulSoup4的学习

和 lxml 一样，Beautiful Soup 也是一个HTML/XML的解析器，主要的功能也是如何解析和提取 HTML/XML 数据。

与 lxml的区别有以下两点：

- ✓ lxml 只会局部遍历，而Beautiful Soup 是基于HTML DOM的，会载入整个文档，解析整个DOM树，因此时间和内存开销都会大很多，所以性能要低于lxml
- ✓ BeautifulSoup 用来解析 HTML 比较简单，API非常人性化，支持CSS选择器、Python标准库中的HTML解析器，也支持 lxml 的 XML解析器

中文文档地址：<https://www.crummy.com/software/BeautifulSoup/bs4/doc/index.zh.html>

安装命令：`pip install bs4`



几大解析工具对比

解析工具	解析速度	使用难度
Beautiful Soup	最慢	最简单
lxml	快	简单
正则表达式	最快	最难



基本使用

```
from bs4 import BeautifulSoup
import lxml

text = '''
<html>
  <body>
    <div>
      <ul>
        <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
        <li class="item2"><a href="https://www.jd.com"> 京东商城</a></li>
        <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
        <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
        <li class="item5"><a href="https://www.google.com">谷歌</a></li>
      </ul>
    </div>
  </body>
</html>
'''
soup = BeautifulSoup(text, 'lxml')
print(type(soup))
print(soup.prettify())
```



```
<class 'bs4.BeautifulSoup'>
<html>
  <body>
    <div>
      <ul>
        <li class="item1">
          <a href="https://geekori.com">
            geekori.com
          </a>
        </li>
        <li class="item2">
          <a href="https://www.jd.com">
            京东商城
          </a>
        </li>
        <li class="item3">
          <a href="https://www.taobao.com">|
            淘宝
          </a>
        </li>
        <li class="item4" value="1234">
          <a href="https://www.microsoft.com">
            微软
          </a>
        </li>
        <li class="item5">
          <a href="https://www.google.com">
            谷歌
          </a>
        </li>
      </ul>
    </div>
  </body>
</html>
```



find_all和find方法

1. find_all: 以**列表** (不管符合条件的节点有几个) 的形式返回所有满足条件的标签

```
liList = soup.find_all('li')
print(liList)
```



```
[<li class="item1"><a href="https://geekori.com"> geekori.com</a></li>,
<li class="item2"><a href="https://www.jd.com"> 京东商城</a></li>,
<li class="item3"><a href="https://www.taobao.com">淘宝</a></li>,
<li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>,
<li class="item5"><a href="https://www.google.com">谷歌</a></li>]
```

2. 可以用limit参数限制提取标签的个数

```
liList = soup.find_all('li', limit=2)
print(liList)
```



```
[<li class="item1"><a href="https://geekori.com"> geekori.com</a></li>,
<li class="item2"><a href="https://www.jd.com"> 京东商城</a></li>]
```

3. 使用find_all的过滤条件

```
liList1 = soup.find_all('li', id='first', class_='item1')
print(liList1)
print('=' * 30)
liList2 = soup.find_all('li', attrs={'id': 'first', 'class': 'item1'})
print(liList2)
```



```
[<li class="item1" id="first"><a href="https://www.jd.com"> 京东商城</a></li>]
=====
[<li class="item1" id="first"><a href="https://www.jd.com"> 京东商城</a></li>]
```

- ✓ 将属性的名字作为关键字参数的名字, 以及属性的值作为关键字参数的值进行过滤
- ✓ 将属性条件放在一个字典里, 传给attrs参数

```
<html>
  <body>
    <div>
      <ul>
        <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
        <li id='first' class="item1"><a href="https://www.jd.com"> 京东商城</a></li>
        <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
        <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
        <li class="item5"><a href="https://www.google.com">谷歌</a></li>
      </ul>
    </div>
  </body>
</html>
```

4. find: 找到第一个满足条件的标签, 只返回一个元素; 用法等同find_all

```
li1 = soup.find('li')
print(li1)
print('=' * 30)
li2 = soup.find('li', id='first', class_='item1')
print(li2)
print('=' * 30)
li3 = soup.find('li', attrs={'class': 'item4', 'value': '1234'})
print(li3)
```

→

```
<li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
=====
<li class="item1" id="first"><a href="https://www.jd.com"> 京东商城</a></li>
=====
<li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
```

5. 通过标签下标的方式获得标签的属性

```
aList= soup.find_all('a')
for a in aList:
    href = a['href']
    print(href)
```

↓

```
https://geekori.com
https://www.jd.com
https://www.taobao.com
https://www.microsoft.com
https://www.google.com
```

```
<html>
  <body>
    <div>
      <ul>
        <li class="item1"><a href="https://geekori.com"> geekori.com</a></li>
        <li id='first' class="item1"><a href="https://www.jd.com"> 京东商城</a></li>
        <li class="item3"><a href="https://www.taobao.com">淘宝</a></li>
        <li class="item4" value="1234"><a href="https://www.microsoft.com">微软</a></li>
        <li class="item5"><a href="https://www.google.com">谷歌</a></li>
      </ul>
    </div>
  </body>
</html>
```

四、数据存储



文件的读写操作

文件的读写模式可以总结为三大类*：读r (read)、写w (write)、追加a (append)

- ✓ r: 只读，该文件必须已经存在
- ✓ rb: 以二进制方式读取文件，该文件必须已经存在
- ✓ w: 只写，打开即默认创建一个新文件，如果文件已存在，则覆盖写（即文件内原始数据会被新写入的数据清空覆盖）
- ✓ wb: 以二进制写方式打开，只能写文件，如果文件不存在，创建该文件；如果文件已存在，则覆盖写
- ✓ a: 追加写，若打开的是已有文件则直接对已有文件操作，若打开文件不存在则创建新文件，只能执行写（追加在后面），不能读

* 更详细内容请参阅 <https://www.cnblogs.com/wujiaqing/p/10690265.html>



CSV文件处理

CSV (Comma-Separated Values, 逗号分隔值) *, 泛指具有以下特征的任何文件:

- ✓ 纯文本, 使用某个字符集, 比如ASCII、Unicode、EBCDIC或GB2312 (简体中文环境) 等
- ✓ 由`记录`组成 (典型的是每一行一条记录)
- ✓ 每条记录被`分隔符`分割为`字段` (典型分割符有逗号、分号或者制表符; 有时分割符可以包括可选的空格)
- ✓ 每条记录都有同样的字段序列

1. 读取CSV文件 --- csv.reader

1	name	author	style	time	publisher	score
2	We Sing.	Jason Mraz	民谣	2008/5/13	Atlantic/WE	9.1
3	Viva La V	Coldplay	摇滚	2008/6/17	Capitol	8.7
4	华丽的冒险	陈绮贞	流行	2005/9/23	艾迴唱片	8.9
5	范特西	周杰伦	流行	2001/9/14	BMG	9.2
6	後。青春	五月天	摇滚	2008/10/23	相信音樂	8.8
7	是时候	孙燕姿	流行	2011/3/8	美妙音乐	8.6
8	Lenka	Lenka	流行	2008/9/23	Epic	8.5
9	Start fr	王若琳	爵士	2008/1/11	SONY BMG唱	8.7
10	旅行的意	陈绮贞	流行	2004/2/2	cheerego. cc	9.2
11	太阳	陈绮贞	流行	2009/1/22	艾迴	8.6
12	We Sing.	Jason Mraz	民谣	2008/5/13	Atlantic/WE	9.1
13	Viva La V	Coldplay	摇滚	2008/6/17	Capitol	8.7
14	华丽的冒险	陈绮贞	流行	2005/9/23	艾迴唱片	8.9
15	范特西	周杰伦	流行	2001/9/14	BMG	9.2
16	後。青春	五月天	摇滚	2008/10/23	相信音樂	8.8
17	是时候	孙燕姿	流行	2011/3/8	美妙音乐	8.6
18	Lenka	Lenka	流行	2008/9/23	Epic	8.5
19	Start fr	王若琳	爵士	2008/1/11	SONY BMG唱	8.7
20	旅行的意	陈绮贞	流行	2004/2/2	cheerego. cc	9.2

```
with open('douban_music.csv', 'r') as fp:
    reader = csv.reader(fp)
    next(reader)
    for x in reader:
        name = x[1]
        score = x[-1]
        info = {
            'name': name,
            'score': score
        }
        print(info)
```

生成reader迭代器

next让遍历对象往下
挪一位, 跳过标题行

迭代读取CSV文件中的
的每一行数据

2. 读取CSV文件 --- csv.DictReader

csv.reader读取方法在获取数据时必须通过下标的方式，这在有的时候不方便，容易出问题。

利用DictReader()方法，可通过标题（关键字）来获取数据

1	name	author	style	time	publisher	score
2	We Sing.	Jason Mraz	民谣	2008/5/13	Atlantic/WEA	9.1
3	Viva La Vida	Coldplay	摇滚	2008/6/17	Capitol	8.7
4	华丽的冒险	陈绮贞	流行	2005/9/23	艾迴唱片	8.9
5	范特西	周杰伦	流行	2001/9/14	BMG	9.2
6	後。青春	五月天	摇滚	2008/10/23	相信音樂	8.8
7	是时候	孙燕姿	流行	2011/3/8	美妙音乐	8.6
8	Lenka	Lenka	流行	2008/9/23	Epic	8.5
9	Start from Me	王若琳	爵士	2008/1/11	SONY BMG唱片	8.7
10	旅行的意义	陈绮贞	流行	2004/2/2	cheerful.co	9.2
11	太阳	陈绮贞	流行	2009/1/22	艾迴	8.6
12	We Sing.	Jason Mraz	民谣	2008/5/13	Atlantic/WEA	9.1
13	Viva La Vida	Coldplay	摇滚	2008/6/17	Capitol	8.7
14	华丽的冒险	陈绮贞	流行	2005/9/23	艾迴唱片	8.9
15	范特西	周杰伦	流行	2001/9/14	BMG	9.2
16	後。青春	五月天	摇滚	2008/10/23	相信音樂	8.8
17	是时候	孙燕姿	流行	2011/3/8	美妙音乐	8.6
18	Lenka	Lenka	流行	2008/9/23	Epic	8.5
19	Start from Me	王若琳	爵士	2008/1/11	SONY BMG唱片	8.7
20	旅行的意义	陈绮贞	流行	2004/2/2	cheerful.co	9.2

```
with open('douban_music.csv', 'r') as fp:
    reader = csv.DictReader(fp)
    for x in reader:
        name = x['name']
        score = x['score']
        info = {
            'name': name,
            'score': score
        }
    print(info)
```

使用DictReader创建的reader对象不会包含标题

Reader是一个迭代器，遍历这个迭代器，返回的是有序字典

```
OrderedDict([('name', 'We Sing. We Dance. We Steal Things.'),
             ('author', 'Jason Mraz'),
             ('style', '民谣'),
             ('time', '2008/5/13'),
             ('publisher', 'Atlantic/WEA'),
             ('score', '9.1')])
```

3. 写入CSV文件 --- csv.writer

```
headers = ['username', 'age', 'height']
values = [
    ['张三', 18, 180],
    ['李四', 20, 170],
    ['王五', 30, 165]
]

with open('classroom.csv', 'w', newline='') as fp:
    writer = csv.writer(fp)
    writer.writerow(headers)
    writer.writerows(values)
```

newline参数克服数据与数据之间的空行

writerow写入一行数据

writerows写入多行数据

这种写入多行的方法要求每个数据只能是tuple类型或者list类型

4. 写入CSV文件 --- csv.DictWriter

```
headers = ['username', 'age', 'height']
values = [
    {'username': '张三', 'age': 18, 'height': 180},
    {'username': '李四', 'age': 20, 'height': 170},
    {'username': '王五', 'age': 30, 'height': 165},
]
with open('classroom1.csv', 'w', newline='') as fp:
    writer = csv.DictWriter(fp, headers)
    writer.writeheader()
    writer.writerows(values)
```

这种写入多行的方法要求每个数据是Dict类型

要将表头传给DictWriter对象

要调用writeheader写入表头

由于在爬虫中存储数据一般是用Dict或者JSON文件，因此csv.DictWriter方法用的比较多



作业6 选取前面五个作业中的两个，将爬取的数据保存在JSON文件中或者CSV文件中

五、动态网页数据抓取

有很多网站显示在页面上的数据并不是一次性从服务端获取的，有一些网站，如图像搜索网站，当滚动条下拉时，有更多的图片显示出来，它们是通过异步的方式不断地从服务端获取的，这就是动态网页（异步数据）

这种异步加载的技术就是AJAX（Asynchronous JavaScript and XML）

```
<script>
(function () {
  if (window._alaEnter && window._alaEnter !== true) {
    window._alaEnter('end');
    var oldOnerror = window.onerror;
    window.onerror = function (msg, source, lineno, colno) {
      try {
        oldOnerror && oldOnerror.apply(window, arguments);
        var str = 'error&source=' + source + '&msg=' + msg;
        window._alaEnter(str);
      } catch (e) {}
    };
  }
})();
</script>
```



```
Headers Preview Response Initiator Timing Cookies
fetchJSON_comment98({productAttr: null,...});
▼ comments: [{id: 15340102058, guid: "5fe70dc46978301eea7cc90e53cfc308",...},...]
  ▶ 0: {id: 15340102058, guid: "5fe70dc46978301eea7cc90e53cfc308",...}
  ▶ 1: {id: 15356856331, guid: "ccdafe902e4b37890f5750729e92e6e7",...}
  ▶ 2: {id: 15343084673, guid: "042f71126101dd9aa8ad241612a8213b",...}
  ▶ 3: {id: 15162308850, guid: "9d95956335ef4da2097a13bdf980624b",...}
  ▶ 4: {id: 15199479054, guid: "d73bf3f94f4b5419185d5ce116771f7e",...}
  ▶ 5: {id: 15286644851, guid: "b8ba728e15a6e59c067af82cbb23a5dc",...}
  ▶ 6: {id: 15361783396, guid: "4a3bb5da955c41862a47e1d90e6ce753",...}
  ▶ 7: {id: 15374942379, guid: "68e4537414b95c4abba1c0de0f677031",...}
  ▶ 8: {id: 15343078549, guid: "a56c74bf29630fa686afc64ea4cbe4c2",...}
  ▶ 9: {id: 15100530349, guid: "20e75ebb65d9e067a26cc4538e0b5339",...}
  csv: "eid=100^tagid=ALL^pid=20001^sku=100009737523^sversion=1001^pageSize
  hotCommentTagStatistics: [{id: "d3c43db0985ec379", name: "完美运行", count: 58
  imageListCount: 500
  jwotestProduct: null
  maxPage: 100
  productAttr: null
  productCommentSummary: {skuId: 100009737523, averageScore: 5, defaultGoodCount
  score: 0
  soType: 5
  testId: "cmt"
  vTagStatistics: null
```



发送请求

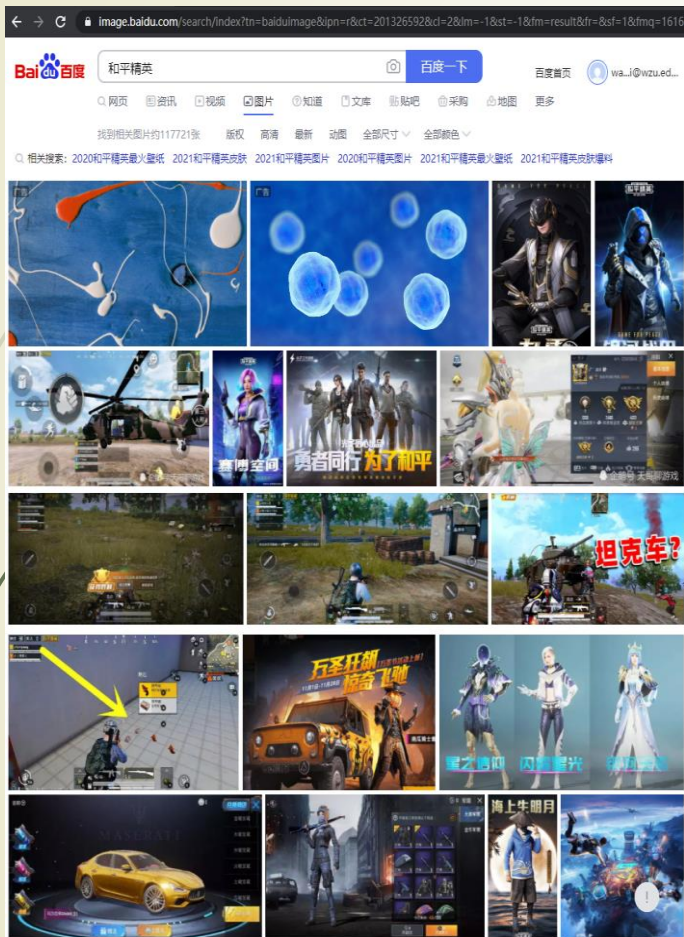
解析响应
(JSON)

渲染页面



动态网页的例子和特点

百度图片



小饭桌资讯



京东评价



注意：通过AJAX技术加载的数据即使添加到当前网页了，但它**并不能在网页源代码中显示**



获取AJAX数据的方式

获取AJAX数据的方式有两种：

- ✓ 直接分析AJAX调用的接口，然后通过代码请求这个接口
- ✓ 使用Selenium+Chromedriver模拟浏览器行为获取数据

两种方式的比较：

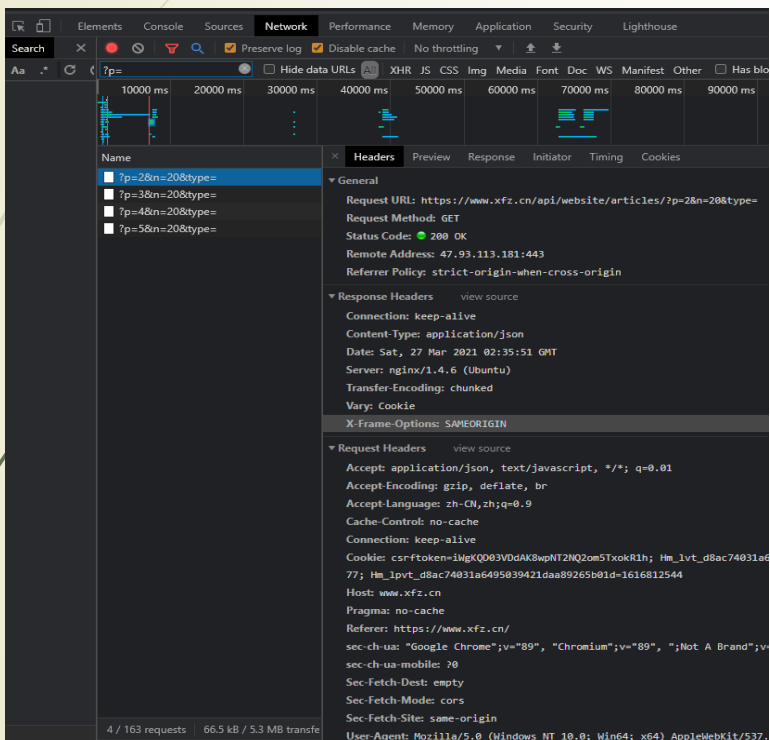
方式	优点	缺点
分析接口	直接可以请求到数据，不需要做一些解析工作，代码量少，性能高	分析接口比较复杂，特别是一些通过JS混淆的接口，要有一定的JS功底；容易被发现是爬虫
selenium	直接模拟浏览器的行为，浏览器能请求到的，使用selenium也能请求到，爬虫更稳定	代码量多，性能低



直接分析接口的方式

1. 爬取小饭桌中的资讯，包括标题、作者、发表时间、内容摘要及内容

✓ Step1. 分析AJAX接口



查看更多

发现“查看更多”规律：



解析响应(JSON)

<https://www.xfz.cn/api/website/articles/?p=2&n=20&type=>
<https://www.xfz.cn/api/website/articles/?p=3&n=20&type=>
<https://www.xfz.cn/api/website/articles/?p=4&n=20&type=>



网页渲染

✓ Step2. 分析文章详情页URL

<https://www.xfz.cn/post/10074.html>

<https://www.xfz.cn/post/10073.html>

<https://www.xfz.cn/post/10072.html>

「易点租」获超5亿人民币E轮融资，源码资本领投

饭桌君 03月16日 融资消息

经纬中国、云时资本和某主权基金跟投。



小饭桌获悉，3月16日，国内知名的办公IT基础设施服务商易点租宣布完成了超5亿人民币的E轮融资，由老股东源码资本领投，经纬中国、云时资本和某主权基金跟投。本轮资金将主要用于核心技术研发与创新，提升数字化服务能力，扩大城市与区域深度覆盖，完善产品和生态建设。其中，源码资本为公司C轮和D轮融资方。

同时，易点租正式宣布品牌升级为“易点云”，持续深耕“办公云”行业，成为企业信赖的办公IT基础设施，推动中国办公室IaaS（基础设施即服务）发展。历经6年的沉淀，易点云从设备的点线面体构建出强大的算力运营能力，在全国范围内建立了实时的基础设施服务网络，可主动为客户进行办公室弹性算力终端部署。

```
{
  "code": 200,
  "data": {
    {
      "uid": 10074,
      "title": "「易点租」获超5亿人民币E轮融资，源码资本领投",
      "photo": "https://static-image.xfz.cn/1615861588_855.jpg",
      "author": @Object{...},
      "is_original": false,
      "article_type": "融资消息",
      "intro": "经纬中国、云时资本和某主权基金跟投。",
      "source": "",
      "time": "2021-03-16 10:26:28",
      "keywords": @Array[1]
    }
  }
}
```

可点击key和

「OTM」获4000万元天使轮融资 创世伙伴CCV领投

饭桌君 03月15日 融资消息

高榕资本跟投。



小饭桌获悉，3月15日，北京——今天，酒旅会员互联网运营商OTM宣布获得4000万元天使轮融资，本轮融资由创世伙伴CCV领投，高榕资本跟投。本轮融资将持续用于技术研发、市场拓展和人才引进。

OTM创始人兼CEO孟令航先生表示：“移动互联网在人们生活中的渗透率越来越高，酒旅行业的预订也不

```
{
  "uid": 10073,
  "title": "「OTM」获4000万元天使轮融资 创世伙伴ccv领投",
  "photo": "https://static-image.xfz.cn/1615774582_954.png",
  "author": @Object{...},
  "is_original": false,
  "article_type": "融资消息",
  "intro": "高榕资本跟投。",
  "source": "",
  "time": "2021-03-15 10:16:24",
  "keywords": @Array[2]
}
```

百福控股强力加持，旗下优选麻辣烫品牌福客进京！

饭桌君 03月13日 其它

福客是一个聚焦高端麻辣烫的连锁餐饮品牌。

3月13日，百福控股旗下优选麻辣烫品牌福客正式进京，北京首店选址为北京昌发展万科广场。福客是一个聚焦高端麻辣烫的连锁餐饮品牌，目前在国内共有近30+家门店，总部位于深圳。



```
{
  "uid": 10072,
  "title": "百福控股强力加持，旗下优选麻辣烫品牌福客进京！",
  "photo": "https://static-image.xfz.cn/1615624955_42.png",
  "author": @Object{...},
  "is_original": false,
  "article_type": "其它",
  "intro": "福客是一个聚焦高端麻辣烫的连锁餐饮品牌。",
  "source": "",
  "time": "2021-03-13 16:42:38",
  "keywords": @Array[1]
}
```

只需要找到资讯文章的id即可找到该文章的详情页，文章id在返回的JSON文本中的uid属性中

Step3. 分析文章详情页内容

xfz.cn/post/10074.html

小饭桌 资讯 创业课堂 在线课堂

「易点租」获超5亿人民币E轮融资，源码资本领投

融资消息 饭桌君 03月16日

“ 经纬中国、云时资本和某主权基金跟投。”



小饭桌获悉，3月16日，国内知名的办公IT基础设施服务商易点租宣布完成了超5亿人民币的E轮融资，由老股东源码资本领投，经纬中国、云时资本和某主权基金跟投。本轮资金将主要用于核心技术研发与创新，提升数字化服务能力，扩大城市与区域深度覆盖，完善产品和生态建设。其中，源码资本为公司C轮和D轮领投方。

同时，易点租正式宣布品牌升级为“易点云”，持续深耕“办公云”行业，成为企业信赖的办公IT基础设施，推动中国办公室IaaS（基础设施即服务）发展。历经6年的沉淀，易点云从设备的点线面体构建出强大的算力运营能力，在全国范围内建立了实时的基础设施服务网络，可主动为客户进行办公室弹性算力终端部署。

//h1[@class="title"]/text()

//span[@class="author-name"]/text()

//span[@class="time"]/text()

//div[@class="content-lead"]/text()

//div[@class="content-detail"]/span/text()

```
<div class="detail-content">
  <h1 class="title">「易点租」获超5亿人民币E轮融资，源码资本领投</h1>
  <div class="author-share">
    <div class="author">
      <span class="author_photo" style="background-image:url(https://static-image.xfz.cn/1552965456_492.jpg-member.photo);"></span>
      <span class="name_time">
        <span class="author-name">饭桌君</span>
        <span class="time">03月16日</span>
      </span>
      <span class="article-type">融资消息</span>
    </div>
    <div class="wxApi"></div>
    <div class="share"></div>
  </div>
  <!-- 导语 -->
  <div class="content-lead"> == $0
    " 经纬中国、云时资本和某主权基金跟投。 "
  </div>
  <!-- 新闻内容 -->
  <div class="content-detail">
    <p style="text-align: center;">
      <span style="color: rgb(63, 63, 63); font-size: 16px;">
        "小饭桌获悉，3月16日，国内知名的办公IT基础设施服务商易点租宣布完成了超5亿人民币的E轮融资，由老股东源码资本领投，经纬中国、云时资本和某主权基金跟投。本轮资金将主要用于核心技术研发与创新，提升数字化服务能力，扩大城市与区域深度覆盖，完善产品和生态建设。其中，源码资本为公司C轮和D轮领投方。 "
      </span>
    </p>
    <p>
      <span style="color: rgb(63, 63, 63); font-size: 16px;">
        "同时，易点租正式宣布品牌升级为“易点云”，持续深耕“办公云”行业，成为企业信赖的办公IT基础设施，推动中国办公室IaaS（基础设施即服务）发展。历经6年的沉淀，易点云从设备的点线面体构建出强大的算力运营能力，在全国范围内建立了实时的基础设施服务网络，可主动为客户进行办公室弹性算力终端部署。 "
      </span>
    </p>
    <p style="text-align: center;"></p>
    <p style="text-align: center;"></p>
  </div>
</div>
```


✓ Step4. 代码分析

```
if __name__ == '__main__':
    BASE_DETAIL_URL = 'https://www.xfz.cn/post/{}.html'
    urls = ['https://www.xfz.cn/api/website/articles/?p={}&n=20&type='.format(str(i))
            for i in range(1, 2)]
    count = 0
    with open('xfz_articles.json', 'w', encoding='utf-8') as fp:
        article_details = []
        for url in urls:
            ids = parse_page(url)
            time.sleep(random.choice(range(1, 5)))
            detail_urls = list(map(lambda x: BASE_DETAIL_URL.format(x), ids))
            for detail_url in detail_urls:
                article_detail = parse_details(detail_url)
                article_details.append(article_detail)
                count += 1
            print('第%d条记录完成了' % count)
            time.sleep(random.choice(range(1, 5)))
        json.dump(article_details, fp, ensure_ascii=False)
        print('文件保存成功!')
```

将元素为字典的列表数据dump到json文件中，ensure_ascii=False可以克服中文乱码问题

返回该url中每一篇文章的id

```
def parse_page(url):
    response = requests.get(url, headers=headers)
    response_json = response.json()
    response_json_data_list = response_json['data']
    ids = []
    for response_json_data in response_json_data_list:
        id = response_json_data['uid']
        ids.append(id)
    return ids
```

获取每一篇文章的详情页url

返回文章详情页信息

```
def parse_details(url):
    response = requests.get(url, headers=headers)
    text = response.text
    html = etree.HTML(text)

    title = html.xpath('//h1[@class="title"]/text()')[0]
    author = html.xpath('//span[@class="author-name"]/text()')[0]
    pub_time = html.xpath('//span[@class="time"]/text()')[0]
    content_lead = html.xpath('//div[@class="content-lead"]/text()')[0].strip()
    content = html.xpath('//div[@class="content-detail"]//span/text()')

    content = ''.join(content)
    img_urls = html.xpath('//div[@class="content-detail"]//img/@src')
    article_detail = {
        'title': title,
        'author': author,
        'pub_time': pub_time,
        'content_lead': content_lead,
        'content': content,
        'img_urls': img_urls,
    }
    return article_detail
```